# Secure but Broken?

Deployment-validated evaluation of LLM security repairs for Kubernetes · progress report, 4 October 2026 · Abdul Hadi (prepared with Claude Code)

## Summary in plain words

We are testing whether AI models (LLMs) that fix security mistakes in Kubernetes files produce fixes that are really safe and still let the application work. Today the whole measuring machine was finished and checked: an online shop (Google's Online Boutique) with 18 planted security mistakes, and a pipeline that deploys every AI fix on a fresh cluster and checks it seven ways. All checks were proven to work before any AI was tested. The first AI runs (DeepSeek V3.2, exploratory, not part of the final results) already show the problem the study is about: on the hardest case the AI produced a fix that every security scanner accepted and that was secure, but it broke the shop completely (93% of requests failed).

| What | Result |
|---|---|
| Planted mistakes (cases), all validated | 18 of 18 |
| Normal speed of the shop on the test machine (10 runs) | p95 11.9 ± 0.9 ms, 0 errors |
| Functional checks catch a broken service | 10 of 10 |
| Test fixes with a known right answer judged correctly | 6 of 6 |
| First AI fixes tried (exploratory) | 7: 6 safe, 1 broke the app |

## What the study asks

| Question | In plain words |
|---|---|
| RQ1 | How often does an AI fix that passes the scanner break the application? |
| RQ2 | How often does it pass the scanner but leave the security problem in place (a deceptive fix)? |
| RQ3 | Which kinds of mistakes and which AI models cause the most broken or deceptive fixes? |
| RQ4 | Does telling the AI what went wrong at runtime (our feedback loop) lead to more fixes that are both safe and working, and at what cost? |

## How one AI fix is judged: seven gates

| Gate | Question | Tool |
|---|---|---|
| L1 | Is the reported security finding gone? | Checkov, KICS, kube-score |
| L2 | Did the fix add any new findings? | same scanners, compared with the clean app |
| L3 | Does Kubernetes accept the files? | kubectl dry-run |
| L4 | Do all parts of the shop start? | fresh kind cluster |
| L5 | Do the shopping journeys work (browse, cart, checkout, email)? | k6 + log check |
| L6 | Is the shop as fast and error-free as normal under traffic? | k6, limit p95 ≤ 14.5 ms, errors ≤ 1% |
| L7 | Is the security problem really gone on the running system? | Kyverno policy + live process / traffic probe |

Verdicts: **SAFE** = all gates pass. **BREAKING** = scanner happy but the app breaks (L3–L6). **DECEPTIVE** = scanner happy, app works, but the security problem is still there (L7). **NOT FIXED** = the scanner still complains, or new problems were added.

## Can we trust the measuring machine?

Before testing any AI, every part of the pipeline was tested on situations where we know the right answer. All measurements now run on a dedicated cloud machine (AWS c6i.2xlarge) because the laptop was too noisy: background programs made the clean shop look broken.

| Check | Laptop | Dedicated cloud VM |
|---|---|---|
| p95 speed of the clean shop | 178 ± 25 ms (5 runs) | 11.9 ± 0.9 ms (10 runs) |
| Clean shop judged as a perfect fix | failed once (machine too busy) | passed |
| Broken services caught by L5 | 10 of 10 | 10 of 10 |
| Cases valid | 18 of 18 | 18 of 18 |
| Known-answer tests | 5 of 6 (laptop noise) | 6 of 6 |

Known-answer tests (hand-made fixes whose correct verdict we know):

| Test fix | Expected | Got | Gate that caught it |
|---|---|---|---|
| safe gold for privileged | SAFE_FIX | SAFE_FIX | none |
| notfixed case unchanged | NOT_FIXED | NOT_FIXED | l1, l2 |
| deceptive allow all netpol | DECEPTIVE_FIX | DECEPTIVE_FIX | l7 |
| breaking tiny memory limit | BREAKING_FIX | BREAKING_FIX | l3 |
| breaking oom runtime | BREAKING_FIX | BREAKING_FIX | l4 |
| breaking cpu starved under load | BREAKING_FIX | BREAKING_FIX | l6 |

## The 18 planted mistakes

| Mistake | Fix class | Services | Valid |
|---|---|---|---|
| added-capabilities | a | cartservice, checkoutservice, frontend | 3/3 |
| allow-privilege-escalation | a | cartservice, checkoutservice, frontend | 3/3 |
| missing-limits | b | cartservice, checkoutservice, frontend | 3/3 |
| netpol | c | cartservice, checkoutservice, productcatalogservice | 3/3 |
| privileged | a | cartservice, checkoutservice, frontend | 3/3 |
| run-as-root | b | cartservice, checkoutservice, frontend | 3/3 |

Fix class: **a** = remove or flip one setting; **b** = choose a value that suits the app; **c** = needs to understand how the services talk to each other. Classification is backed by the Kubernetes Pod Security Standards and the NSA/CISA Kubernetes Hardening Guidance.

## First AI results (exploratory, not counted)

These runs only check that the cases are hard enough and the pipeline works with a real AI. They are excluded from the paper's numbers, and the cases were not changed because of them (decision D-025).

| Case | Class | Verdict | Failed gates | What the AI did |
|---|---|---|---|---|
| privileged-frontend | a | SAFE_FIX | none | Turned privileged off and restored the escalation block (= gold) |
| allow-privilege-escalation-checkoutservice | a | SAFE_FIX | none | Repair identical to the hardened original |
| added-capabilities-cartservice | a | SAFE_FIX | none | Repair identical to the hardened original |
| run-as-root-frontend | b | SAFE_FIX | none | Repair identical to the hardened original |
| missing-limits-cartservice | b | SAFE_FIX | none | Repair identical to the hardened original |
| netpol-cartservice | c | BREAKING_FIX | l5, l6 | Allowed traffic out only to Redis and blocked DNS: 93% of requests failed, p95 6,007 ms |
| privileged-checkoutservice | a | SAFE_FIX | none | Repair identical to the hardened original |

**The broken fix is the paper's point in one example.** On the network-rules case the AI wrote a rule that correctly kept outsiders out (security check passed) and passed every scanner, but it also blocked the cart service from looking up the database's name (DNS). The shop failed almost completely. A study that only looks at scanners would have counted this as a success.

## Feedback loops on the hard case

We then let the AI try again with feedback, on the same network case. First run (before fixes):

| Condition | Iterations | Verdict per iteration | Final |
|---|---|---|---|
| C1 | 1 | BREAKING_FIX | BREAKING_FIX |
| C2-template | 5 | BREAKING_FIX → NOT_FIXED → BREAKING_FIX → NOT_FIXED → NOT_FIXED | NOT_FIXED |
| C2-LLM | 5 | BREAKING_FIX → BREAKING_FIX → BREAKING_FIX → BREAKING_FIX → INVALID_OUTPUT | INVALID_OUTPUT |

C1 stopped after one try because the scanner was happy (exactly the blind spot of earlier research). Both C2 variants failed for reasons that were our design flaws: the AI moderator's messages were wrongly rejected (it cited evidence as [E1, E2]), and the real cause was in the logs of healthy services, which we did not collect. Both were fixed (D-028).

Second run, after the fixes:

| Condition | Iterations | Verdict per iteration | Final |
|---|---|---|---|
| C1 | 1 | BREAKING_FIX | BREAKING_FIX |
| C2-template | 4 | BREAKING_FIX → BREAKING_FIX → BREAKING_FIX → SAFE_FIX | SAFE_FIX |
| C2-LLM | 3 | BREAKING_FIX → BREAKING_FIX → SAFE_FIX | SAFE_FIX |

With the fixes, **both feedback loops repaired the case**: the AI moderator's messages were accepted first time, it correctly diagnosed DNS failures, and DeepSeek fixed the rule (SAFE after 3 tries with the LLM moderator, 4 with the fixed template). The scanner-only loop (C1) still stopped at the broken fix. This is one exploratory example, not a result. **New gap found:** the final 'safe' fix in the template run deleted the namespace's default deny-all rule. Our gates passed it because the target service stayed protected, but deleting that rule weakens protection for the whole namespace. This must be decided before the pilot.

## Findings so far

| # | Finding | Why it matters |
|---|---|---|
| F1 | The shop hides 2 of 10 broken services (pages still load). | Simple 'page loads' checks miss real breakage. |
| F3 | A privileged container can look exactly like a safe one if you only check effective permissions. | Security checks must look deeper; ours do. |
| F4 | None of the three scanners notices a network rule that lets everyone in. | Deceptive fixes can pass every scanner. |
| F6 | A CPU-starved shop passes scanners, starts and works, but is about 55× too slow under traffic (p95 657 ms vs 11.9 ms normal, on the VM). | Only load testing finds it. |
| F7 | Our own security check once gave a false alarm on root containers; caught and fixed. | Checking the checker matters. |
| F8 | AI fix: secure, scanner-clean, but blocked DNS and broke the shop. | Real example of 'secure but broken'. |
| F9 | The run-as-root cases leave a hint of the right user ID. | Kept; reported as a limitation. |
| F10 | The AI invented a service that exists only in newer versions of the shop. | Caught by L2. |
| F11 | Runtime feedback only helps if the evidence contains the real cause. | Evidence design is part of the method. |
| F12 | With complete evidence, both runtime-feedback loops fixed the broken network case; the scanner-only loop did not. | First sign of the effect RQ4 tests (one example). |
| F13 | A 'safe' fix deleted the namespace-wide deny-all rule; our checks only protect the target service. | Open gap: should removing other security controls count as deceptive? |

## Problems we found in our own setup and fixed

| Problem | Fix |
|---|---|
| Laptop too noisy to measure speed | Dedicated cloud VM; all measurements redone there |
| Security check could not run inside root containers | Different debug profile; all 18 cases re-checked |
| Feedback checker rejected valid citations | Accept [E1, E2] style; tests added |
| Root cause hidden in healthy services' logs | Error lines from every service added to the evidence |
| Windows line endings would change file fingerprints | Line endings pinned; fresh-clone test passes |
| Journal times estimated instead of read from the clock | Correction entry; times now from the clock |

## Key decisions made

| Decision | What we decided |
|---|---|
| Models | Claude Sonnet 4.6, GPT-OSS 120B, DeepSeek V3.2; Claude Haiku 4.5 as moderator (newest models blocked on the account) |
| Scope | 6 mistake types × 3 services = 18 cases; more later as a separate batch |
| Difficulty | No invented easy/medium/hard labels; fix classes a/b/c defined up front, cited sources |
| Measuring machine | Dedicated AWS VM; scaling with identical VMs + a job queue, not Spot instances |
| Prompt | Run both with and without 'keep the application working' |
| Verdict rules | New scanner problems = not fixed; leaving out objects is judged by the gates |
| Load test | Retest once if it fails; fail only if both fail |

## Cost so far

AWS Cost Explorer lags by up to a day and showed only USD 0.03 for October when this report was built. Estimate for 4 October: measurement VM about 7 hours at USD 0.34/hour (about USD 2.40) plus about 30 AI calls of roughly 8,000 tokens in and out each (expected well under USD 2). Everything is paid from the AWS credits (about USD 528). The exact figure will be read from Cost Explorer tomorrow.

## Next steps

| Step | What |
|---|---|
| 0 | Decide on gap F13 (removed security controls elsewhere) and clean the noisy log lines in the evidence |
| 1 | Worker machine image + small fleet (plan and cost shown before creating) |
| 2 | Freeze the prompt (both wordings) and the protocol draft |
| 3 | Pilot: 2 models × 3 cases × conditions × 3 repeats; measure real cost and variance |
| 4 | Full experiment, then EKS check and analysis |

Everything is recorded: decisions (docs/decisions.md), journal (docs/journal.md), every shell command (docs/command-log/), all raw results in a versioned S3 bucket, and the steps to reproduce (docs/REPRODUCE.md).